K6 Performance Testing

Complete Guide for QA Engineers

From zero to production-ready performance testing

QA Portal · Internal Use · Version 1.0 · October 2026

Table of Contents

1. What is Performance Testing?
2. Why Do We Do Performance Testing?
3. Types of Performance Tests
4. Prerequisites — What You Need Before You Start
5. Tools Overview
6. Study Materials (Placeholders)
7. Setting Up Your Environment
8. K6 Basics — How It Works
9. Writing Your First K6 Test
10. Understanding K6 Script Structure
11. Checks, Thresholds & Metrics
12. Generic Templates
13. Viewing Results in the Terminal
14. Reporting with Grafana
15. Running Tests — Common Commands
16. Best Practices
17. Common Mistakes to Avoid
18. Troubleshooting
19. Quick Reference Cheat Sheet

1. What is Performance Testing?

Performance testing checks how well your application behaves under load. Instead of one user clicking through your app, you simulate many users hitting it at the same time — and you measure what happens.

Think of it like this:

- Functional testing = Does the login button work?
- Performance testing = Does the login button still work when 500 people click it at the same time?

2. Why Do We Do Performance Testing?

- Find out how many users your app can handle before it slows down or crashes
- Catch performance regressions before they reach production
- Prove to stakeholders that the system is ready for launch
- Identify the weak point (database, API, third-party service) under load
- Give confidence to the team before high-traffic events (launches, sales, etc.)

3. Types of Performance Tests

You do not need all of these on every project. Start with Load Testing and add others as needed.

Test Type	What It Does	When to Use
Load Test	Simulates expected normal traffic	Before every release
Stress Test	Pushes beyond normal limits to find the breaking point	Before major launches
Spike Test	Sudden huge burst of users, then drop back	Black Friday, flash sales
Soak Test	Sustained load over a long period (hours)	Detecting memory leaks
Smoke Test	Very small load — just to verify the script runs	After writing a new script

4. Prerequisites — What You Need Before You Start

Knowledge Prerequisites

You do not need to be a developer, but the following will help:

- ☐ [ ] Basic understanding of HTTP (what a GET/POST request is)
- ☐ [ ] Know what an API endpoint is (e.g., GET /api/users)
- ☐ [ ] Basic JavaScript syntax — K6 uses JavaScript

- ☐ [ ] Know the difference between a request and a response
- ☐ [ ] Understand HTTP status codes (200=OK, 404=Not Found, 500=Server Error)
- ☐ [ ] Know what JSON looks like

Access Prerequisites

Before starting a test on any project, confirm:

- ☐ [ ] Get the base URL of the environment you will test (staging, not production!)
- ☐ [ ] Get authentication credentials (API key, token, username/password)
- ☐ [ ] Confirm with the team that it is OK to run load tests on this environment
- ☐ [ ] Know the expected user load (how many real users the app should handle)
- ☐ [ ] Get the list of critical API endpoints to test
- ☐ [ ] Make sure the environment is staging/test — never production

> ⚠️ **Important**
>
> Always get written approval before running performance tests. Load tests can slow down or crash a server.

Hardware Prerequisites

- A machine running macOS, Windows, or Linux
- At least 4 GB RAM (8 GB recommended)
- Stable internet connection (if testing against a remote environment)

5. Tools Overview

Tool	Purpose	Required?
K6	Runs your performance tests	Yes — required
Node.js	Not required to run K6, but useful for JS tooling	Optional
Grafana	Visual dashboard to display test results	Recommended
InfluxDB	Time-series database that stores K6 results for Grafana	If using Grafana
Docker	Easiest way to run InfluxDB + Grafana together	Recommended
Postman	For exploring and understanding the API before writing tests	Recommended
VS Code	Code editor for writing test scripts	Recommended

6. Study Materials

> ℹ️ **Note**
>
> Replace each placeholder below with the actual link or attachment before publishing this document.

Required Reading Before Writing Your First Test

- ☐ [ ] [PLACEHOLDER — K6 Getting Started Guide] — Official K6 docs introduction
- ☐ [ ] [K6 Performance testing – Fariha Notes] — Quick handbook
- ☐ [ ] [PLACEHOLDER — JavaScript Basics for QA Engineers]
- ☐ [ ] [PLACEHOLDER — What is an API?] — REST API concepts for testers

Video Tutorials

- ☐ [ ] [PLACEHOLDER — K6 Intro Video] — 15-minute overview of K6
- ☐ [ ] [PLACEHOLDER — Load Testing Fundamentals] — VUs, ramp-up, steady state

Internal Resources

- ☐ [ ] [PLACEHOLDER — Company API Documentation]
- ☐ [ ] [PLACEHOLDER — Staging Environment Access Guide]
- ☐ [ ] [PLACEHOLDER — Performance Test Results Archive]

7. Setting Up Your Environment

Step 1 — Install K6

macOS (Homebrew)

```
brew install k6
```

Windows (Chocolatey)

```
choco install k6
```

Windows (winget)

```
winget install k6 --source winget
```

Linux (Debian/Ubuntu)

```
sudo gpg -k
sudo gpg --no-default-keyring \
  --keyring /usr/share/keyrings/k6-archive-keyring.gpg \
  --keyserver hkp://keyserver.ubuntu.com:80 \
  --recv-keys C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] \
  https://dl.k6.io/deb stable main" | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt-get update && sudo apt-get install k6
```

Verify installation

```
k6 version
# Expected output: k6 v0.53.0 (or newer)
```

Step 2 — Install Docker

Download Docker Desktop from the official website and install it. Verify it is running:

```
docker --version
```

Step 3 — Set Up InfluxDB + Grafana

Create a file called docker-compose.yml anywhere on your machine and paste the following:

```
version: '3'

services:
  influxdb:
    image: influxdb:1.8
    ports:
      - "8086:8086"
    environment:
      - INFLUXDB_DB=k6
    networks:
      - k6

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    networks:
      - k6
    depends_on:
      - influxdb

networks:
  k6:
```

Start the containers:

```
docker-compose up -d
# Grafana will be available at: http://localhost:3000
```

Step 4 — Configure Grafana

1. Open http://localhost:3000 in your browser
2. Go to Configuration → Data Sources → Add data source
3. Select InfluxDB
4. Set the URL to: http://influxdb:8086
5. Set the Database to: k6
6. Click Save & Test — you should see "Data source connected"

Step 5 — Import K6 Dashboard

7. In Grafana, go to Dashboards → Import
8. Enter dashboard ID: 2587 (official K6 dashboard)
9. Select your InfluxDB data source
10. Click Import — your dashboard is ready

Step 6 — Project Folder Structure

Create a folder for your performance tests with this structure:

```
my-project-performance-tests/
├── tests/
│   ├── smoke-test.js
│   ├── load-test.js
│   ├── stress-test.js
│   └── soak-test.js
├── config/
│   └── environments.js
├── utils/
│   └── helpers.js
└── README.md
```

8. K6 Basics — How It Works

K6 simulates Virtual Users (VUs). Each VU runs your test script from top to bottom, over and over, for the test duration.

Key terms you need to know:

Term	What It Means
VU (Virtual User)	A simulated user running your script
Duration	How long the test runs
Ramp-up	Gradually increasing VUs at the start
Steady state	Holding VU count at peak load
Ramp-down	Gradually decreasing VUs at the end
Iteration	One full run of your script by one VU
Threshold	A pass/fail rule (e.g., p95 must be < 500ms)
Check	An assertion inside your script (e.g., status code is 200)

9. Writing Your First K6 Test

Create a file called smoke-test.js and add this code:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

// Test configuration
export const options = {
  vus: 1,          // 1 virtual user
  duration: '10s', // run for 10 seconds
};

// Your test scenario
export default function () {
  // Make a GET request
  const response = http.get('https://test-api.example.com/health');

  // Check that the response is correct
  check(response, {
    'status is 200':          (r) => r.status === 200,
    'response time < 500ms':  (r) => r.timings.duration < 500,
  });

  // Wait 1 second between iterations (simulates real user behavior)
  sleep(1);
}
```

Run it from the terminal:

```
k6 run smoke-test.js
```

> ℹ️ **Note**
>
> That's it — you just ran your first performance test!

10. Understanding K6 Script Structure

Every K6 script has five possible sections:

Section	When It Runs	Purpose
Imports	N/A (static)	Bring in K6 modules (http, check, sleep, etc.)
options (export)	Before test starts	Configure VUs, stages, thresholds
setup() (export)	Once at the start	Login, create test data, get auth token
default() (export)	Every VU iteration	The main test scenario — what each user does
teardown() (export)	Once at the end	Clean up test data, log results

Full annotated script structure:

```
// 1. IMPORTS
import http from 'k6/http';
import { check, sleep, group } from 'k6';

// 2. OPTIONS — configure how the test runs
export const options = {
  stages: [
    { duration: '1m', target: 50 },  // ramp up to 50 VUs
    { duration: '3m', target: 50 },  // stay at 50 VUs
    { duration: '1m', target: 0  },  // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% of requests < 500ms
    http_req_failed:   ['rate<0.01'], // < 1% failures
  },
};

// 3. SETUP — runs ONCE before the test
export function setup() {
  const loginRes = http.post('https://api.example.com/auth/login',
    JSON.stringify({ username: 'testuser', password: 'testpass' }),
    { headers: { 'Content-Type': 'application/json' } }
  );
  return { token: loginRes.json('token') };
}
```

```
// 4. DEFAULT — runs for each VU, each iteration
export default function (data) {
  const headers = { 'Authorization': `Bearer ${data.token}` };
  const res = http.get('https://api.example.com/users', { headers });
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}

// 5. TEARDOWN — runs ONCE after the test
export function teardown(data) {
  console.log('Test finished.');
}
```

11. Checks, Thresholds & Metrics

Checks

Checks are pass/fail assertions inside your script. They do NOT stop the test if they fail — they just record the failure rate.

```
check(response, {
  'status is 200':            (r) => r.status === 200,
  'body contains user':       (r) => r.body.includes('user'),
  'response time < 1s':       (r) => r.timings.duration < 1000,
  'content-type is JSON':     (r) =>
r.headers['Content-Type'].includes('application/json'),
});
```

Thresholds

Thresholds are pass/fail criteria for the whole test. If a threshold is breached, K6 exits with a non-zero exit code — useful for CI/CD pipelines.

```
export const options = {
  thresholds: {
    'http_req_duration': ['p(95)<500'],   // 95th percentile < 500ms
    'http_req_duration': ['avg<200'],     // average < 200ms
    'http_req_failed':   ['rate<0.01'],   // less than 1% failures
    'checks':            ['rate>0.99'],   // 99% of checks must pass
  },
};
```

Key Metrics (tracked automatically)

Metric	What It Means
`http_req_duration`	Total time for the HTTP request (most important metric)
`http_req_failed`	Rate of failed requests (non-2xx or network error)
`http_reqs`	Total number of requests made
`vus`	Current number of active virtual users
`iterations`	Number of times the default function completed
`data_received`	Amount of data received from the server
`data_sent`	Amount of data sent to the server

Understanding Percentiles

When you see p(95) it means: "95% of requests were faster than this value."

Percentile	Meaning
p(50)	Median — half were faster, half were slower
p(90)	90% of requests were below this time
p(95)	Commonly used as the SLA threshold — recommended default
p(99)	Worst-case realistic scenario (excludes extreme outliers)

12. Generic Templates

> ℹ️ **Note**
>
> All templates below use placeholder values. Search for "REPLACE" and swap in your actual URLs, endpoints, and credentials.

Template 1 — Smoke Test

Purpose: Verify the script works with minimal load before scaling up.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://YOUR-STAGING-URL.com';  // REPLACE THIS

export const options = {
  vus: 1,
  duration: '30s',
  thresholds: {
    http_req_failed:   ['rate<0.01'],
    http_req_duration: ['p(95)<2000'],
  },
};

export default function () {
  const response = http.get(`${BASE_URL}/api/health`);  // REPLACE endpoint

  check(response, {
    'status is 200':    (r) => r.status === 200,
    'response < 2s':    (r) => r.timings.duration < 2000,
  });

  sleep(1);
}
```

Template 2 — Load Test (Normal Traffic)

Purpose: Simulate normal expected traffic. Use this for every release.

```
import http from 'k6/http';
import { check, sleep, group } from 'k6';

const BASE_URL      = 'https://YOUR-STAGING-URL.com';  // REPLACE THIS
const EXPECTED_USERS = 100;                             // REPLACE: your normal user
count

export const options = {
  stages: [
    { duration: '2m', target: EXPECTED_USERS * 0.5 }, // ramp up to 50%
    { duration: '5m', target: EXPECTED_USERS },        // full load
```

```
    { duration: '10m', target: EXPECTED_USERS },       // hold for 10 minutes
    { duration: '2m', target: 0 },                     // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500', 'p(99)<1500'],
    http_req_failed:   ['rate<0.01'],
    checks:            ['rate>0.99'],
  },
};

export default function () {
  group('Browse Products', function () {
    const res = http.get(`${BASE_URL}/api/products`);  // REPLACE
    check(res, { 'list loaded': (r) => r.status === 200 });
    sleep(2);
  });

  group('View Product Detail', function () {
    const res = http.get(`${BASE_URL}/api/products/1`);  // REPLACE
    check(res, { 'detail loaded': (r) => r.status === 200 });
    sleep(1);
  });

  sleep(1);
}
```

Template 3 — Load Test with Authentication

Purpose: Load test for endpoints that require a logged-in user.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://YOUR-STAGING-URL.com';         // REPLACE THIS
const USERNAME = __ENV.TEST_USERNAME || 'testuser';       // pass via -e flag
const PASSWORD = __ENV.TEST_PASSWORD || 'testpass';       // pass via -e flag

export const options = {
  stages: [
    { duration: '1m', target: 20 },
    { duration: '5m', target: 50 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<1000'],
    http_req_failed:   ['rate<0.01'],
  },
};

// Runs ONCE — get auth token
export function setup() {
  const loginRes = http.post(
    `${BASE_URL}/api/auth/login`,           // REPLACE login endpoint
    JSON.stringify({ username: USERNAME, password: PASSWORD }),
    { headers: { 'Content-Type': 'application/json' } }
```

```
  );
  check(loginRes, { 'login successful': (r) => r.status === 200 });
  return { token: loginRes.json('token') }; // REPLACE 'token' with actual field
}

export default function (data) {
  const headers = {
    'Authorization':  `Bearer ${data.token}`,
    'Content-Type':   'application/json',
  };

  const res = http.get(`${BASE_URL}/api/dashboard`, { headers }); // REPLACE
  check(res, { 'dashboard loaded': (r) => r.status === 200 });
  sleep(2);
}
```

Template 4 — Stress Test

Purpose: Push the system beyond normal load to find the breaking point.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL     = 'https://YOUR-STAGING-URL.com';  // REPLACE THIS
const NORMAL_USERS = 100;                              // REPLACE: normal user count

export const options = {
  stages: [
    { duration: '2m',  target: NORMAL_USERS },       // normal load
    { duration: '5m',  target: NORMAL_USERS },       // hold
    { duration: '2m',  target: NORMAL_USERS * 2 },   // 2x normal
    { duration: '5m',  target: NORMAL_USERS * 2 },   // hold at 2x
    { duration: '2m',  target: NORMAL_USERS * 3 },   // 3x normal
    { duration: '5m',  target: NORMAL_USERS * 3 },   // hold at 3x
    { duration: '2m',  target: 0 },                  // recovery
  ],
  thresholds: {
    http_req_failed: ['rate<0.10'], // allow up to 10% failures in stress test
  },
};

export default function () {
  const res = http.get(`${BASE_URL}/api/health`);  // REPLACE
  check(res, { 'service responding': (r) => r.status < 500 });
  sleep(1);
}
```

Template 5 — Spike Test

Purpose: Simulate a sudden traffic spike (product launch, email blast).

```
import http from 'k6/http';
import { check, sleep } from 'k6';
```

```
const BASE_URL   = 'https://YOUR-STAGING-URL.com';  // REPLACE THIS
const PEAK_USERS = 500;                              // REPLACE: spike count

export const options = {
  stages: [
    { duration: '1m',   target: 10 },          // baseline
    { duration: '30s',  target: PEAK_USERS },   // sudden spike
    { duration: '3m',   target: PEAK_USERS },   // hold at peak
    { duration: '30s',  target: 10 },           // drop back
    { duration: '2m',   target: 10 },           // recover
    { duration: '1m',   target: 0 },
  ],
  thresholds: {
    http_req_failed:   ['rate<0.05'],
    http_req_duration: ['p(95)<2000'],
  },
};

export default function () {
  const res = http.get(`${BASE_URL}/api/products`);  // REPLACE
  check(res, { 'not server error': (r) => r.status < 500 });
  sleep(1);
}
```

Template 6 — Soak Test

Purpose: Run sustained load for a long time to detect slow memory leaks or gradual degradation.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL     = 'https://YOUR-STAGING-URL.com';  // REPLACE THIS
const NORMAL_USERS = 50;                               // REPLACE: moderate count

export const options = {
  stages: [
    { duration: '5m', target: NORMAL_USERS },   // ramp up
    { duration: '2h', target: NORMAL_USERS },   // REPLACE: soak duration (2 hours)
    { duration: '5m', target: 0 },              // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed:   ['rate<0.01'],
  },
};

export default function () {
  const res = http.get(`${BASE_URL}/api/products`);  // REPLACE
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(2);
}
```

Template 7 — POST Request

Purpose: Test endpoints that create or update data.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://YOUR-STAGING-URL.com';  // REPLACE THIS

export const options = {
  stages: [
    { duration: '1m', target: 20 },
    { duration: '3m', target: 50 },
    { duration: '1m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<1000'],
    http_req_failed:   ['rate<0.01'],
  },
};

export default function () {
  const payload = JSON.stringify({
    name:  `Test User ${Date.now()}`,       // unique name per iteration
    email: `test${Date.now()}@example.com`, // REPLACE with your fields
    role:  'user',
  });

  const headers = {
    'Content-Type':   'application/json',
    'Authorization':  'Bearer YOUR_TOKEN_HERE',  // REPLACE
  };

  const res = http.post(
    `${BASE_URL}/api/users`,  // REPLACE with your POST endpoint
    payload,
    { headers }
  );

  check(res, {
    'created successfully':  (r) => r.status === 201,
    'returns user id':       (r) => r.json('id') !== undefined, // REPLACE 'id'
  });

  sleep(1);
}
```

Environment Config Helper

Create config/environments.js to manage multiple environments cleanly:

```
// config/environments.js
export function getConfig() {
  const env = __ENV.TEST_ENV || 'staging';
```

```
  const configs = {
    local: {
      baseUrl:       'http://localhost:3000',
      expectedUsers: 10,
      thresholds:    { p95: 2000, failRate: 0.05 },
    },
    staging: {
      baseUrl:       'https://staging.your-company.com',  // REPLACE
      expectedUsers: 100,
      thresholds:    { p95: 500, failRate: 0.01 },
    },
  };

  if (!configs[env]) throw new Error(`Unknown environment: ${env}`);
  return configs[env];
}
```

Usage in your test:

```
import { getConfig } from '../config/environments.js';

const config = getConfig();

export const options = {
  stages: [
    { duration: '2m', target: config.expectedUsers },
    { duration: '5m', target: config.expectedUsers },
    { duration: '1m', target: 0 },
  ],
  thresholds: {
    http_req_duration: [`p(95)<${config.thresholds.p95}`],
    http_req_failed:   [`rate<${config.thresholds.failRate}`],
  },
};

export default function () {
  http.get(`${config.baseUrl}/api/health`);
  sleep(1);
}
```

13. Viewing Results in the Terminal

After running a test, K6 shows a summary in the terminal. Here is what each line means:

```
✓ status is 200
  ✓ response time < 500ms

  checks.........................: 100.00%  ✓ 4500   ✗ 0
  http_req_duration..............: avg=145ms  min=100ms  med=130ms
                                   max=890ms  p(90)=220ms  p(95)=285ms
  http_req_failed................: 0.00%    ✓ 0      ✗ 4500
  http_reqs......................: 4500     6.74/s
  vus............................: 1        min=1    max=50

  ✓ http_req_duration............: p(95)=285ms < 500ms   PASS
  ✓ http_req_failed..............: 0.00% < 1.00%         PASS
```

Line	What It Tells You
checks	How many assertions passed (✓) vs failed (✗)
http_req_duration	Response time stats — focus on p(95) as your key number
http_req_failed	Percentage of requests that failed
http_reqs	Total requests made and requests per second (throughput)
vus	Min and max active virtual users during the test
Threshold lines	PASS/FAIL verdict per threshold you defined

14. Reporting with Grafana (Recommended)

Stream K6 Results to Grafana in Real-Time

Add the --out flag when running your test:

```
k6 run --out influxdb=http://localhost:8086/k6 load-test.js
```

What You See in Grafana

- Real-time VU count over time
- Request rate (requests per second)
- Response time trends (p90, p95, p99)
- Error rate over time
- Request duration breakdown by endpoint

Exporting for Reports

To save a PNG screenshot for a test report:

11. Open Grafana at http://localhost:3000
12. Navigate to your K6 dashboard
13. Set the time range to match your test window (top-right corner)
14. Take a screenshot OR use Grafana Share → Export as PDF

Save Results as JSON (for archiving)

```
k6 run --out json=results/load-test-$(date +%Y%m%d).json load-test.js
```

15. Running Tests — Common Commands

```
# Basic run
k6 run smoke-test.js

# Send results to Grafana via InfluxDB
k6 run --out influxdb=http://localhost:8086/k6 load-test.js

# Pass environment variables
k6 run -e TEST_ENV=staging -e TEST_USERNAME=testuser -e TEST_PASSWORD=secret load-
test.js

# Override VUs and duration from command line
k6 run --vus 10 --duration 30s smoke-test.js

# Save output to JSON file
k6 run --out json=results.json load-test.js

# Show detailed request/response info (for debugging)
k6 run --http-debug smoke-test.js

# Export a summary report
k6 run --summary-export=summary.json load-test.js
```

16. Best Practices

Before Running

- Always run a smoke test first — 1 VU, 30 seconds — before scaling up. This catches broken scripts before they generate real load.
- Never test against production. Always use staging or a dedicated performance test environment.
- Get approval from your team before generating significant load.
- Know your baseline first — record the response time before any load is applied.

While Writing Tests

- Add sleep() between steps to simulate real user behavior. Without it, VUs will hammer the server as fast as possible.
- Use realistic data — vary product IDs, user IDs, search terms, etc.
- Use group() to organize steps. This makes reports easier to read.
- Set thresholds that match your SLA. If the business says pages must load in under 2 seconds, set p(95)<2000.
- Use environment variables for credentials and URLs — never hardcode passwords.

Reporting

- Always note which environment you tested against
- Always note the date and time of the test
- Include a baseline comparison (before and after)
- Keep Grafana dashboard screenshots with each test run
- Document any anomalies or unexpected findings

17. Common Mistakes to Avoid

Mistake	Why It Is a Problem	Fix
No sleep() between requests	Unrealistic, hammers server unfairly	Add sleep(1) or sleep(Math.random() * 3)
Testing against production	Risk of crashing real users' experience	Always use staging
No smoke test first	Wastes load run time with a broken script	Always smoke test first
Hardcoded credentials	Security risk, hard to change per environment	Use __ENV.VARIABLE_NAME
Very short test duration	Misses steady-state and warm-up behavior	5–10 minutes minimum for load tests
No thresholds set	Cannot auto-detect pass/fail in CI/CD	Always define thresholds
No ramp-up stage	Unrealistic; acts like a spike test	Always include a ramp-up stage

Only looking at average	Average hides slow outliers	Use p(95) as primary metric

18. Troubleshooting

Many request failures or WARN Request Failed

- Check that the URL is correct
- Check that the environment is up and accessible
- Verify auth token/headers are correct
- Run with --http-debug to see full request and response details

Script runs but no data appears in Grafana

- Confirm InfluxDB and Grafana containers are running: docker ps
- Confirm you ran with --out influxdb=http://localhost:8086/k6
- Check InfluxDB is accessible: open http://localhost:8086 in browser

"Too many open files" error

This means your OS limit on concurrent connections is too low. Run:

```
ulimit -n 250000
```

"ERRO[0001] GoError: dial tcp: lookup..." error

K6 cannot resolve the hostname. Check your URL is correct and that your machine has internet/network access.

19. Quick Reference Cheat Sheet

K6 options block

```
export const options = {
  vus:        50,          // fixed VU count
  duration:   '5m',        // fixed duration
  iterations: 1000,        // fixed total iterations (alternative to duration)
  stages: [
    { duration: '1m', target: 20 },
    { duration: '5m', target: 50 },
    { duration: '1m', target: 0  },
  ],
  thresholds: {
    'http_req_duration': ['p(95)<500'],
    'http_req_failed':   ['rate<0.01'],
    'checks':            ['rate>0.99'],
  },
};
```

HTTP methods

```
const h = { 'Content-Type': 'application/json', 'Authorization': 'Bearer TOKEN' };
const b = JSON.stringify({ key: 'value' });

http.get   ('https://example.com/api/items',    { headers: h });
http.post  ('https://example.com/api/items', b, { headers: h });
http.put   ('https://example.com/api/items/1', b, { headers: h });
http.patch ('https://example.com/api/items/1', b, { headers: h });
http.del   ('https://example.com/api/items/1', null, { headers: h });
```

Checks

```
check(response, {
  'status 200':          (r) => r.status === 200,
  'has body':            (r) => r.body.length > 0,
  'field exists':        (r) => r.json('fieldName') !== undefined,
  'fast response':       (r) => r.timings.duration < 500,
});
```

Sleep

```
sleep(1);                          // wait exactly 1 second
sleep(Math.random() * 3 + 1);     // wait 1–4 seconds (more realistic)
```

Environment variables

```
// Run command:
k6 run -e BASE_URL=https://staging.example.com -e TOKEN=abc123 test.js

// In script:
const BASE_URL = __ENV.BASE_URL || 'http://localhost:3000';
const TOKEN    = __ENV.TOKEN;
```

Recommended thresholds by endpoint type

Endpoint Type	p95 Target	Failure Rate
Health check	< 100ms	< 0.1%
Simple GET (list / search)	< 300ms	< 1%
Single resource GET	< 200ms	< 1%
POST (create)	< 500ms	< 1%
PUT / PATCH (update)	< 500ms	< 1%
Authentication / Login	< 1000ms	< 0.5%
File upload	< 5000ms	< 2%
Report / complex query	< 3000ms	< 2%

Document version: 1.0 · Last updated: October 2026 · Owner: [PLACEHOLDER — QA Team Lead]

For questions or updates contact: [PLACEHOLDER — qa-team@your-company.com]